
SEZIONE 1: DOMANDE TEORICHE

Domanda 1.1 - Ricorrenze e Metodo di Sostituzione (6 punti)

Consegna: Si dimostri che la ricorrenza che segue ha soluzione $T(n) = \Theta(n)$:

$$T(n) = (2/3)T(n-1) + 2n$$

Soluzione:

Dimostrazione per $O(n)$: Vogliamo dimostrare che esistono costanti $c > 0$ e n_0 tali che $T(n) \leq cn$ per ogni $n \geq n_0$.

Procediamo per induzione su n .

Caso base: Assumiamo $T(1) = c_1$ per qualche costante c_1 . Scegliamo $c \geq c_1$.

Passo induttivo: Supponiamo per ipotesi induttiva che $T(n-1) \leq c(n-1)$. Allora:

$$\begin{aligned} T(n) &= (2/3)T(n-1) + 2n \\ &\leq (2/3)c(n-1) + 2n && \text{[per ipotesi induttiva]} \\ &= (2/3)cn - (2/3)c + 2n \\ &= cn - (2/3)c + 2n - (1/3)cn \\ &= cn + 2n - (1/3)cn - (2/3)c \\ &= cn + n(2 - c/3) - (2/3)c \end{aligned}$$

Affinché $T(n) \leq cn$, dobbiamo avere:

$$n(2 - c/3) - (2/3)c \leq 0$$

Scegliamo $c = 12$. Allora:

$$n(2 - 4) - 8 = -2n - 8 \leq 0$$

che vale per ogni $n \geq 1$.

Quindi $T(n) = O(n)$.

Dimostrazione per $\Omega(n)$: Vogliamo dimostrare che esistono costanti $d > 0$ e n_0 tali che $T(n) \geq dn$ per ogni $n \geq n_0$.

Procediamo per induzione su n .

Caso base: $T(1) = c_1$. Scegliamo $d \leq c_1$.

Passo induttivo: Supponiamo $T(n-1) \geq d(n-1)$. Allora:

$$\begin{aligned} T(n) &= (2/3)T(n-1) + 2n \\ &\geq (2/3)d(n-1) + 2n && \text{[per ipotesi induttiva]} \\ &= (2/3)dn - (2/3)d + 2n \\ &= dn + 2n - (1/3)dn - (2/3)d \\ &= dn + n(2 - d/3) - (2/3)d \end{aligned}$$

Affinché $T(n) \geq dn$, dobbiamo avere:

$$n(2 - d/3) - (2/3)d \geq 0$$

Scegliamo $d = 3$. Allora:

$$n(2 - 1) - 2 = n - 2 \geq 0$$

che vale per ogni $n \geq 2$.

Quindi $T(n) = \Omega(n)$.

Conclusione: Poiché $T(n) = O(n)$ e $T(n) = \Omega(n)$, concludiamo che $T(n) = \Theta(n)$.

Domanda 1.2 - Alberi Binari di Ricerca (7 punti)

Consegna: Dare la definizione di albero binario di ricerca. Specificare l'albero ottenuto inserendo, con la procedura vista a lezione, a partire da un albero vuoto, i nodi aventi le seguenti chiavi: 10, 5, 3, 15, 17, 12. Si supponga che dall'albero così ottenuto si cancelli il nodo con chiave 5 e si indichi l'albero ottenuto. Sia per gli inserimenti che per la cancellazione, motivare sinteticamente il risultato ottenuto.

Soluzione:

Definizione di Albero Binario di Ricerca (BST): Un albero binario di ricerca è un albero binario in cui ogni nodo x ha i campi:

- $x.key$: la chiave del nodo
- $x.left$: puntatore al figlio sinistro
- $x.right$: puntatore al figlio destro
- $x.p$: puntatore al padre

La proprietà fondamentale dei BST è: Per ogni nodo x , tutte le chiavi nel sottoalbero sinistro di x sono $\leq x.key$ e tutte le chiavi nel sottoalbero destro di x sono $\geq x.key$.

Sequenza di Inserimenti:

1. Inserisco 10: diventa la radice

```
10
```

2. Inserisco 5: $5 < 10$, quindi va a sinistra

```
  10
 /
5
```

3. Inserisco 3: $3 < 10$ (vado a sinistra), $3 < 5$ (vado ancora a sinistra)

```
  10
 /
 5
 /
3
```

4. Inserisco 15: $15 > 10$, quindi va a destra

```
  10
 /  \
 5    15
 /
3
```

5. Inserisco 17: $17 > 10$ (vado a destra), $17 > 15$ (vado ancora a destra)

```
  10
 /  \
 5    15
 /      \
3        17
```

6. Inserisco 12: $12 > 10$ (vado a destra), $12 < 15$ (vado a sinistra)

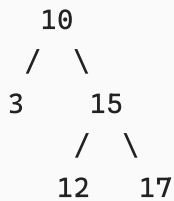
```
  10
 /  \
 5    15
 /      \
3        17
 /  \
12  17
```

Cancellazione del nodo 5:

Il nodo 5 ha un solo figlio (il nodo 3). Secondo la procedura di cancellazione:

- Sostituiamo il nodo 5 con il suo unico figlio 3
- Il sottoalbero radicato in 3 viene collegato direttamente al padre di 5 (il nodo 10)

Albero finale:



Motivazione: Quando cancelliamo un nodo con un solo figlio, sostituiamo semplicemente il nodo con il suo figlio. Questo preserva la proprietà BST perché il figlio rispetta già la relazione d'ordine con il padre del nodo cancellato.

SEZIONE 2: DIVIDE ET IMPERA

Esercizio 2.1 - Indici Stabili (10 punti)

Consegna: Dato un array $A[1,n]$ di interi, un indice i si dice stabile se $A[i] = i$. Realizzare una procedura $\text{stab}(A, n)$ che, dato in input un array $A[1,n]$ di interi distinti ordinato in modo crescente, ritorna un indice stabile, se esiste, e ritorna 0 altrimenti. Dimostrarne la correttezza e valutarne la complessità.

Soluzione:

Idea dell'algoritmo: Utilizziamo un approccio divide-et-impera simile alla ricerca binaria. Osserviamo che se $A[i] > i$, allora per ogni $j > i$ vale $A[j] > j$ (perché l'array è ordinato e contiene elementi distinti). Analogamente, se $A[i] < i$, per ogni $j < i$ vale $A[j] < j$.

Pseudocodice:

```
1  stab(A, n)
2    return stab_rec(A, 1, n)

3  stab_rec(A, p, r)
4    if p > r
5      return 0
6    q = ⌊(p + r)/2⌋
7    if A[q] = q
8      return q
9    else if A[q] > q
```

```

10     return stab_rec(A, p, q-1)
11     else
12     return stab_rec(A, q+1, r)

```

Dimostrazione di Correttezza:

Dimostriamo per induzione sulla dimensione dell'intervallo $[p,r]$ che:

- Se esiste un indice stabile in $A[p,r]$, la funzione lo ritorna
- Altrimenti ritorna 0

Caso base: Se $p > r$, l'intervallo è vuoto, quindi correttamente ritorniamo 0.

Passo induttivo: Assumiamo l'invariante vero per intervalli di dimensione $< (r - p + 1)$.

Sia $q = \lfloor (p + r)/2 \rfloor$. Abbiamo tre casi:

1. **$A[q] = q$:** L'indice q è stabile, lo ritorniamo (corretto).
2. **$A[q] > q$:**
 - Per ogni $j > q$: $A[j] \geq A[q] + (j - q) > q + (j - q) = j$
 - Quindi nessun indice $j > q$ può essere stabile
 - Se esiste un indice stabile, deve essere in $[p, q-1]$
 - Per ipotesi induttiva, $\text{stab_rec}(A, p, q-1)$ è corretto
3. **$A[q] < q$:**
 - Per ogni $j < q$: $A[j] \leq A[q] - (q - j) < q - (q - j) = j$
 - Quindi nessun indice $j < q$ può essere stabile
 - Se esiste un indice stabile, deve essere in $[q+1, r]$
 - Per ipotesi induttiva, $\text{stab_rec}(A, q+1, r)$ è corretto

Analisi di Complessità:

La ricorrenza è:

$$T(n) = T(n/2) + \theta(1)$$

Applicando il Master Theorem con $a=1$, $b=2$, $f(n)=\Theta(1)$:

- $n^{(\log_b a)} = n^{(\log_2 1)} = n^0 = 1$
- $f(n) = \Theta(1) = \Theta(n^{(\log_2 1)})$

Siamo nel caso 2 del Master Theorem con $k=0$:

$$T(n) = \theta(n^{(\log_2 1)} \log^{\theta} n) = \theta(\log n)$$

Conclusione: La complessità è $\Theta(\log n)$.

Esercizio 2.2 - Gap in Array (9 punti)

Consegna: Dato un array di interi $A[1..n]$, chiamiamo gap un indice $i \in [1, n)$ tale che $A[i+1] - A[i] > 1$.

1. Mostrare per induzione su n che un array $A[1..n]$ tale che $A[n] - A[1] \geq n$ (quindi $n \geq 2$) contiene almeno un gap.
2. Fornire lo pseudocodice di una procedura ricorsiva divide et impera gap che dato un array $A[1..n]$ tale che $A[n] - A[1] \geq n$ restituisce un gap in A .
3. Valutare la complessità della funzione, utilizzando il master theorem.

Soluzione:

Parte 1: Dimostrazione per Induzione

Vogliamo dimostrare che se $A[n] - A[1] \geq n$, allora esiste almeno un gap.

Caso base ($n = 2$): Se $A[2] - A[1] \geq 2$, allora $A[2] - A[1] > 1$, quindi $i = 1$ è un gap. ✓

Passo induttivo ($n > 2$): Assumiamo la proprietà vera per array di lunghezza $< n$.

Consideriamo $A[1..n]$ con $A[n] - A[1] \geq n$.

Caso 1: Se $A[n] - A[n-1] > 1$, allora $i = n-1$ è un gap. ✓

Caso 2: Se $A[n] - A[n-1] \leq 1$, allora:

$$\begin{aligned} A[n-1] - A[1] &= (A[n] - A[1]) - (A[n] - A[n-1]) \\ &\geq n - 1 \end{aligned}$$

Per ipotesi induttiva, $A[1..n-1]$ contiene un gap, che è anche un gap per $A[1..n]$. ✓

Parte 2: Pseudocodice

```
1  gap(A, n)
2    return gap_rec(A, 1, n)

3  gap_rec(A, p, r)
4    // Precondizione:  $p < r$  e  $A[r] - A[p] \geq r - p + 1$ 
5    if  $r - p = 1$ 
6      return p    //  $A[r] - A[p] > 1$  per precondizione
7     $q = \lfloor (p + r)/2 \rfloor$ 
8    if  $A[q] - A[p] \geq q - p + 1$ 
9      return gap_rec(A, p, q)
10   else
11     return gap_rec(A, q, r)
```

Dimostrazione di Correttezza:

Invariante: Se la preconditione vale, la funzione ritorna un gap.

Caso base: Se $r - p = 1$, allora per preconditione $A[r] - A[p] \geq 2$, quindi $A[r] - A[p] > 1$, e p è un gap.

Passo ricorsivo: Sia $q = \lfloor (p + r)/2 \rfloor$.

Caso 1: Se $A[q] - A[p] \geq q - p + 1$:

- La preconditione vale per $[p, q]$
- Per ipotesi ricorsiva, $\text{gap_rec}(A, p, q)$ ritorna un gap

Caso 2: Se $A[q] - A[p] < q - p + 1$:

- Allora $A[r] - A[q] = (A[r] - A[p]) - (A[q] - A[p]) \geq (r - p + 1) - (q - p) = r - q + 1$
- La preconditione vale per $[q, r]$
- Per ipotesi ricorsiva, $\text{gap_rec}(A, q, r)$ ritorna un gap

Parte 3: Analisi di Complessità

La ricorrenza è:

$$T(n) = T(n/2) + \theta(1)$$

Per il Master Theorem (caso 2 con $k=0$):

$$T(n) = \theta(\log n)$$

SEZIONE 3: PROGRAMMAZIONE DINAMICA

Esercizio 3.1 - Ricorrenza su Stringhe (9 punti)

Consegna: Data una stringa $X = (x_1, x_2, \dots, x_n)$, si consideri la seguente quantità $\ell(i, j)$, definita per $1 \leq i \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } i = j - 1 \\ 2 + \ell(i+1, j-1) & \text{se } (i < j-1) \wedge (x_i = x_j) \\ \sum_{k=i}^{j-1} [\ell(i, k) + \ell(k+1, j)] & \text{se } (i < j-1) \wedge (x_i \neq x_j) \end{cases}$$

(a) Scrivere una coppia di algoritmi $\text{INIT_L}(X)$ e $\text{REC_L}(X, i, j)$ per il calcolo memoizzato di $\ell(1, n)$.

(b) Determinarne la complessità al caso migliore $T_{\text{best}}(n)$, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

Soluzione:

Parte (a): Algoritmi Memoizzati

```
1  INIT_L(X)
2    n = X.length
3    for i = 1 to n do
4      L[i, i] = 1
5      for j = i+1 to n do
6        L[i, j] = -1
7    for i = 1 to n-1 do
8      L[i, i+1] = 2
9    return REC_L(X, 1, n)

10 REC_L(X, i, j)
11   if L[i, j] ≠ -1
12     return L[i, j]
13   if X[i] = X[j]
14     L[i, j] = 2 + REC_L(X, i+1, j-1)
15   else
16     sum = 0
17     for k = i to j-1 do
18       sum = sum + REC_L(X, i, k) + REC_L(X, k+1, j)
19     L[i, j] = sum
20   return L[i, j]
```

Dimostrazione di Correttezza:

L'algoritmo implementa direttamente la definizione ricorsiva, utilizzando la memoizzazione per evitare ricalcoli. La correttezza segue dalla correttezza della definizione e dal fatto che ogni sottoproblema viene calcolato esattamente una volta.

Parte (b): Analisi Complessità Caso Migliore

Il caso migliore si verifica quando **tutti i caratteri sono uguali** ($X = \text{"aaaa...a"}$).

In questo caso:

- Ogni coppia (i, j) con $i < j-1$ soddisfa $X[i] = X[j]$
- La linea 14 viene sempre eseguita (mai il loop delle linee 17-18)
- Per ogni coppia (i, j) viene fatto esattamente un confronto alla linea 13

Numero di sottoproblemi:

Numero di coppie (i, j) con $1 \leq i \leq j \leq n = n(n+1)/2$

Costo per sottoproblema nel caso migliore:

- Linea 13: 1 confronto
- Linea 14: chiamate ricorsive (costo già contato per quei sottoproblemi)
- Linee 11-12, 20: costo 0 (nessun confronto)

Complessità totale caso migliore:

$$T_{\text{best}}(n) = \Theta(n^2)$$

Ogni sottoproblema viene calcolato una volta con 1 confronto, e ci sono $\Theta(n^2)$ sottoproblemi.

Esercizio 3.2 - Longest Common Substring (11 punti)

Consegna: Una longest common substring di due stringhe X e Y è una sottostringa di X e di Y di lunghezza massima. Si vuole progettare un algoritmo efficiente per calcolare la lunghezza di una longest common substring. Per semplicità si assuma che entrambe le stringhe di input abbiano stessa lunghezza n .

(a) Qual è la complessità dell'algoritmo esaustivo che analizza tutte le possibili sottostringhe comuni?

(b) Assumendo di conoscere un algoritmo che determina se una stringa di m caratteri è sottostringa di un'altra stringa di n caratteri in tempo $O(m+n)$, come si può modificare l'algoritmo del punto precedente per renderlo più efficiente?

(c) Progettare un algoritmo di programmazione dinamica più efficiente di quello del punto precedente. Sono richiesti relazione di ricorrenza sulle lunghezze (senza dimostrazione) e algoritmo bottom-up.

Soluzione:

Parte (a): Algoritmo Esaustivo

Per ogni sottostringa di X :

- Generare tutte le sottostringhe di X : $\Theta(n^2)$ sottostringhe
- Per ciascuna, verificare se è sottostringa di Y : $O(n)$ nel caso peggiore

Complessità totale: $\Theta(n^3)$

Parte (b): Miglioramento con Algoritmo $O(m+n)$

Strategia migliorata:

- Generare tutte le sottostringhe di X: $\Theta(n^2)$
- Per ciascuna sottostringa s di lunghezza |s|, verificare se è in Y: $O(|s| + n) = O(n)$

Complessità totale: $\Theta(n^2)$

Il miglioramento deriva dal fatto che l'algoritmo di matching $O(m+n)$ è lineare rispetto alla somma delle lunghezze.

Parte (c): Programmazione Dinamica

Relazione di Ricorrenza:

Definiamo $L[i, j]$ = lunghezza della longest common substring che termina in $X[i]$ e $Y[j]$.

```
L[i, j] = {
    0                se i = 0 o j = 0
    L[i-1, j-1] + 1  se X[i] = Y[j]
    0                se X[i] ≠ Y[j]
}
```

La soluzione è: $\max\{L[i, j] : 1 \leq i, j \leq n\}$

Osservazione critica: A differenza della LCS (Longest Common Subsequence), qui quando i caratteri non coincidono resettiamo a 0, perché una substring deve essere continua.

Algoritmo Bottom-Up:

```
1  LCS_Length(X, Y, n)
2  for i = 0 to n do
3      L[i, 0] = 0
4  for j = 0 to n do
5      L[0, j] = 0
6  maxLength = 0
7  for i = 1 to n do
8      for j = 1 to n do
9          if X[i] = Y[j]
10             L[i, j] = L[i-1, j-1] + 1
11             maxLength = max(maxLength, L[i, j])
12         else
13             L[i, j] = 0
14  return maxLength
```

Dimostrazione di Correttezza:

Invariante dei loop: Dopo l'iterazione (i, j), $L[i, j]$ contiene la lunghezza della longest common substring che termina esattamente in posizione i di X e posizione j di Y.

Correttezza dell'invariante:

- **Caso base ($i=0$ o $j=0$):** Correttamente impostato a 0 (linee 2-5)
- **Caso $X[i] = Y[j]$:** La longest substring che termina in (i, j) è quella che terminava in $(i-1, j-1)$ più il carattere corrente (linea 10)
- **Caso $X[i] \neq Y[j]$:** Non può esserci una substring che termina in entrambe le posizioni (linea 13)

Analisi di Complessità:

Spazio: $\Theta(n^2)$ per la tabella L

Tempo:

- Inizializzazione: $\Theta(n)$ (linee 2-5)
- Loop principale: $\Theta(n^2)$ (linee 7-13)
 - Ogni iterazione: $\Theta(1)$

Complessità totale: $\Theta(n^2)$ tempo, $\Theta(n^2)$ spazio

Miglioramento dello spazio: Si può ridurre a $\Theta(n)$ mantenendo solo due righe della tabella alla volta.

SEZIONE 4: ALBERI BINARI DI RICERCA

Esercizio 4.1 - avgTree (9 punti)

Consegna: Realizzare una funzione `avgTree(T)` che dato un albero binario `T` con chiavi numeriche, verifica se, per ogni nodo che abbia discendenti, la chiave del nodo è maggiore o uguale della media delle chiavi dei discendenti e ritorna conseguentemente un valore booleano (la radice dell'albero è `T.root` e ogni nodo `x` ha i campi `x.left`, `x.right` e `x.key`).

Soluzione:

Idea dell'algoritmo: Effettuiamo una visita ricorsiva dell'albero raccogliendo per ogni sottoalbero:

- Un booleano che indica se la proprietà è soddisfatta
- La somma delle chiavi nel sottoalbero
- Il numero di nodi nel sottoalbero

Pseudocodice:

```
1  avgTree(T)
2    valid, numNodes, sum = avgTreeRec(T.root)
3    return valid
```

```

4  avgTreeRec(x)
5    // Ritorna: (booleano, numero_nodi, somma_chiavi)
6    if x = nil
7        return (true, 0, 0)
8
9    // Visita ricorsiva dei sottoalberi
10   validL, numL, sumL = avgTreeRec(x.left)
11   validR, numR, sumR = avgTreeRec(x.right)
12
13   // Calcolo informazioni per il nodo corrente
14   numNodes = numL + numR + 1
15   sum = sumL + sumR + x.key
16
17   // Verifica della condizione
18   numDescendants = numL + numR
19   if numDescendants = 0
20       // Nodo foglia: sempre valido
21       valid = true
22   else
23       // Nodo interno: verifica media
24       sumDescendants = sumL + sumR
25       avgDescendants = sumDescendants / numDescendants
26       valid = validL and validR and (x.key ≥ avgDescendants)
27
28   return (valid, numNodes, sum)

```

Dimostrazione di Correttezza:

Invariante: Per ogni nodo x , $\text{avgTreeRec}(x)$ ritorna una tripla (v, n, s) dove:

- $v = \text{true} \Leftrightarrow$ il sottoalbero radicato in x soddisfa la proprietà avgTree
- n = numero di nodi nel sottoalbero radicato in x
- s = somma delle chiavi nel sottoalbero radicato in x

Caso base ($x = \text{nil}$):

- L'albero vuoto soddisfa banalmente la proprietà
- Ha 0 nodi e somma 0
- Ritorna $(\text{true}, 0, 0)$ ✓

Passo induttivo ($x \neq \text{nil}$): Assumiamo l'invariante vero per i sottoalberi sinistro e destro.

1. Le chiamate ricorsive (linee 10-11) ritornano le informazioni corrette per i sottoalberi per ipotesi induttiva
2. Calcolo di numNodes e sum (linee 14-15):
 - numNodes include tutti i nodi dei sottoalberi più il nodo corrente ✓
 - sum include tutte le chiavi dei sottoalberi più la chiave corrente ✓

3. Verifica della proprietà (linee 19-26):

- **Se numDescendants = 0:** x è una foglia, la proprietà è soddisfatta banalmente ✓
- **Se numDescendants > 0:**
 - La proprietà è soddisfatta se:
 - I sottoalberi soddisfano la proprietà (validL and validR)
 - E $x.key \geq \text{media dei discendenti}$
 - Questo è esattamente ciò che verifica la linea 26 ✓

Analisi di Complessità:

Ricorrenza:

$$T(n) = T(k) + T(n-k-1) + \Theta(1)$$

dove k è il numero di nodi nel sottoalbero sinistro.

Caso peggiore: $T(n) = T(n-1) + \Theta(1) = \Theta(n)$ (albero degenero) **Caso medio:** $T(n) = 2T(n/2) + \Theta(1) = \Theta(n \log n)$ (albero bilanciato)

In ogni caso, visitiamo ogni nodo esattamente una volta con lavoro $\Theta(1)$ per nodo.

Complessità: $\Theta(n)$ dove n è il numero di nodi dell'albero.

Spazio: $O(h)$ per lo stack di ricorsione, dove h è l'altezza dell'albero.

Esercizio 4.2 - Arricchimento BST con Numero Foglie (9 punti)

Consegna: Realizzare un arricchimento degli alberi binari di ricerca che permetta di ottenere per ogni nodo x, in tempo costante, il numero delle foglie nel sottoalbero radicato in x.

Indicare quali campi occorre aggiungere ai nodi. Fornire il codice per la funzione leaves(x) che restituisce il numero delle foglie nel sottoalbero radicato in x e per la procedura di inserimento di un nodo insert(T, z). Per entrambe valutare la complessità.

Soluzione:

Arricchimento della struttura:

Ogni nodo x ha un nuovo campo:

- **x.numLeaves:** numero di foglie nel sottoalbero radicato in x

Definizione ricorsiva:

```

x.numLeaves = {
    1                                se x è una foglia (x.left = nil e
x.right = nil)
    x.left.numLeaves + x.right.numLeaves altrimenti
}

```

Convenzione: T.nil.numLeaves = 0

Funzione leaves(x):

```

1  leaves(x)
2    if x = nil
3        return 0
4    else
5        return x.numLeaves

```

Complessità: $\Theta(1)$ - accesso diretto al campo.

Procedura insert(T, z):

```

1  insert(T, z)
2    y = nil
3    x = T.root
4
5    // Discesa dell'albero e aggiornamento numLeaves
6    while x ≠ nil do
7        y = x
8        // Se x era una foglia, ora non lo è più
9        if x.left = nil and x.right = nil
10           x.numLeaves = 1 // Sarà aggiornato dopo
11
12       if z.key < x.key
13           x = x.left
14       else
15           x = x.right
16
17       // Inserimento del nuovo nodo
18       z.p = y
19       z.left = nil
20       z.right = nil
21       z.numLeaves = 1 // z è una foglia
22
23       if y = nil
24           T.root = z
25       else if z.key < y.key
26           y.left = z
27       else
28           y.right = z

```

```

29
30 // Aggiornamento numLeaves lungo il cammino
31 x = z.p
32 while x ≠ nil do
33     if x.left = nil and x.right = nil
34         x.numLeaves = 1
35     else
36         leftLeaves = (x.left ≠ nil) ? x.left.numLeaves : 0
37         rightLeaves = (x.right ≠ nil) ? x.right.numLeaves : 0
38         x.numLeaves = leftLeaves + rightLeaves
39     x = x.p

```

Dimostrazione di Correttezza:

Invariante della fase di discesa (linee 6-15): Il puntatore x discende l'albero seguendo la proprietà BST.

Invariante della fase di aggiornamento (linee 32-39): Per ogni nodo x visitato risalendo dall'inserimento alla radice:

- $x.numLeaves$ viene aggiornato correttamente come somma delle foglie dei suoi sottoalberi
- Se x è diventato un nodo interno (aveva 0 figli, ora ne ha 1), $x.numLeaves$ viene calcolato correttamente
- Se x era già un nodo interno, $x.numLeaves$ viene incrementato di 1

Correttezza finale: Dopo l'esecuzione:

1. Il nuovo nodo z è inserito correttamente come foglia (linee 17-28) ✓
2. Tutti i nodi lungo il cammino dalla radice a z hanno $x.numLeaves$ aggiornato correttamente (linee 32-39) ✓
3. Gli altri nodi mantengono valori corretti (non modificati) ✓

Analisi di Complessità:

Fase di discesa: $O(h)$ dove h è l'altezza dell'albero **Fase di inserimento:** $\Theta(1)$ **Fase di aggiornamento:** $O(h)$ - risalita lungo il cammino

Complessità totale: $O(h)$

- **Caso migliore (albero bilanciato):** $O(\log n)$
- **Caso peggiore (albero degenerare):** $O(n)$

Spazio aggiuntivo: $\Theta(1)$ per nodo (un campo aggiuntivo)

SEZIONE 5: MAX-HEAP

Esercizio 5.1 - Intersezione di Min-Heap (9 punti)

Consegna: Realizzare una funzione `intersect(A1, A2, n)` che dati due array di interi `A1` e `A2`, organizzati a min-heap, con capacità `n`, restituisce un nuovo array `A`, ancora organizzato a min-heap, che contiene l'intersezione dei valori contenuti in `A1` e `A2`. Nel caso gli array contengano più occorrenze dello stesso valore `v`, l'intersezione mantiene il numero minimo di occorrenze di `v` (ad es. se `A1` contiene i valori 1,2,2,2 e `A2` contiene i valori 1,1,2,2 allora `A` conterrà 1,2,2). Valutarne la complessità.

Soluzione:

Idea dell'algoritmo:

1. Estraiamo elementi dai due min-heap in ordine crescente (tramite `extract-min`)
2. Quando troviamo valori uguali in entrambi gli heap, li aggiungiamo all'intersezione
3. Costruiamo il risultato come array ordinato, che è automaticamente un min-heap valido

Pseudocodice:

```
1  intersect(A1, A2, n)
2    // Inizializzazione
3    A = nuovo array di capacità n
4    size = 0
5
6    // Copia dimensioni effettive degli heap
7    size1 = A1.heapsize
8    size2 = A2.heapsize
9
10   // Estrazione ordinata e confronto
11   while size1 > 0 and size2 > 0 do
12     v1 = A1[1]
13     v2 = A2[1]
14
15     if v1 = v2
16       // Valore comune: aggiungi all'intersezione
17       A[size + 1] = v1
18       size = size + 1
19       extractMin(A1)
20       size1 = size1 - 1
21       extractMin(A2)
22       size2 = size2 - 1
23
24     else if v1 < v2
25       // v1 non può essere nell'intersezione
26       extractMin(A1)
27       size1 = size1 - 1
28
29     else // v1 > v2
```



```
30      // v2 non può essere nell'intersezione
31      extractMin(A2)
32      size2 = size2 - 1
33
34  A.heapsize = size
35  return A
```

Dimostrazione di Correttezza:

Proprietà dei min-heap: Gli elementi vengono estratti in ordine crescente.

Invariante del loop (linee 11-32):

- Tutti i valori già aggiunti ad A appartengono all'intersezione di A1 e A2
- Tutti i valori $< \min(v1, v2)$ sono stati processati correttamente
- A è un array ordinato in modo crescente

Correttezza dell'invariante:

1. Caso $v1 = v2$ (linee 15-22):

- Il valore $v1 = v2$ appare in entrambi gli heap
- Lo aggiungiamo all'intersezione ✓
- Estraiamo da entrambi gli heap per mantenere il minimo delle occorrenze ✓

2. Caso $v1 < v2$ (linee 24-27):

- $v1$ non può apparire in A2 perché tutti gli elementi rimasti in A2 sono $\geq v2 > v1$
- Correttamente escludiamo $v1$ dall'intersezione ✓

3. Caso $v1 > v2$ (linee 29-32):

- Simmetricamente, $v2$ non può apparire in A1 ✓

Perché A è un min-heap: Poiché inseriamo gli elementi in ordine crescente, l'array A risultante è ordinato. Un array ordinato in ordine crescente soddisfa automaticamente la proprietà di min-heap: ogni elemento è $\leq$ dei suoi figli.

Analisi di Complessità:

Sia:

- $n1$ = dimensione effettiva di A1
- $n2$ = dimensione effettiva di A2

Costo del loop:

- Numero massimo di iterazioni: $\min(n1, n2)$
- Ogni iterazione:
 - Confronto: $\Theta(1)$
 - `extractMin`: $O(\log n)$ ciascuno

- Inserimento in A: $\Theta(1)$

Caso peggiore:

- Tutti gli elementi di entrambi gli heap vengono estratti
- Numero totale di `extractMin`: $O(n_1 + n_2)$
- Costo totale: $O((n_1 + n_2) \log n)$

Caso migliore:

- Gli heap sono disgiunti e uno dei due si svuota rapidamente
- Costo: $O(\min(n_1, n_2) \log n)$

Complessità: $O((n_1 + n_2) \log n)$ dove $n = \max(n_1, n_2)$

Spazio: $\Theta(n)$ per l'array risultato.

Ottimizzazione possibile: Se vogliamo solo il risultato come array ordinato senza mantenerlo come heap, possiamo evitare le operazioni di `extractMin` e usare semplicemente l'accesso alla radice seguito dallo spostamento degli elementi (complessità migliora a $O(n_1 + n_2)$).

SEZIONE 6: ALGORITMI GREEDY

Esercizio 6.1 - Minimizzazione Tempo Completamento (10 punti)

Consegna: Abbiamo n programmi da eseguire sul nostro computer. Ogni programma j , dove $j \in \{1, 2, \dots, n\}$, ha lunghezza l_j , che rappresenta la quantità di tempo richiesta per la sua esecuzione.

Dato un ordine di esecuzione $\sigma = j_1, j_2, \dots, j_n$ dei programmi (cioè, una permutazione di $\{1, 2, \dots, n\}$), il tempo di completamento $C_{ji}(\sigma)$ del j_i -esimo programma è dato quindi dalla somma delle lunghezze dei programmi $j_1, j_2, \dots, j_i$.

L'obiettivo è trovare un ordine di esecuzione σ che minimizza la somma dei tempi di completamento di tutti i programmi, cioè $\sum_i C_{ji}(\sigma)$.

- (a) Dare un semplice algoritmo greedy per questo problema, e valutarne la complessità.
- (b) Dimostrare la proprietà di scelta greedy dell'algoritmo del punto (a), cioè che esiste un ordine di esecuzione ottimo σ^* che contiene la scelta greedy.

Soluzione:**Parte (a): Algoritmo Greedy**

Idea: Ordiniamo i programmi per lunghezza crescente.

Scelta greedy: Eseguiamo per primo il programma con lunghezza minima.

Pseudocodice:

```
1  MinCompletionTime(L[1..n])
2    // Input: array L di lunghezze dei programmi
3    // Output: permutazione ottima  $\sigma$ 
4
5    // Ordina i programmi per lunghezza crescente
6     $\sigma$  = Sort(L)
7
8    return  $\sigma$ 
```

Versione dettagliata:

```
1  MinCompletionTime(programs[1..n])
2    // Ogni programs[i] ha campo programs[i].length
3
4    // Ordina per lunghezza crescente
5    for i = 1 to n-1 do
6      for j = i+1 to n do
7        if programs[j].length < programs[i].length
8          swap(programs[i], programs[j])
9
10   return programs
```

Analisi di Complessità:

Con sorting efficiente (MergeSort/HeapSort):

- Ordinamento: $\Theta(n \log n)$
- **Complessità totale:** $\Theta(n \log n)$

Con bubble sort (come nel pseudocodice sopra):

- Ordinamento: $\Theta(n^2)$
- **Complessità totale:** $\Theta(n^2)$

La versione efficiente usa $\Theta(n \log n)$.

Parte (b): Dimostrazione Proprietà Scelta Greedy

Vogliamo dimostrare che esiste una soluzione ottima che inizia con il programma di lunghezza minima.

Teorema (Scelta Greedy): Sia $l_{\min}$ il programma con lunghezza minima. Esiste un ordine di esecuzione ottimo σ^* che esegue $l_{\min}$ per primo.

Dimostrazione (Exchange Argument):

Sia $\sigma^* = (p_1, p_2, \dots, p_n)$ una soluzione ottima qualunque.

Caso 1: Se $p_1 = l_{\min}$, abbiamo finito. ✓

Caso 2: Se $p_1 \neq l_{\min}$, sia $p_k = l_{\min}$ per qualche $k > 1$.

Costruiamo σ' scambiando p_1 e p_k :

$$\sigma' = (l_{\min}, p_2, \dots, p_{k-1}, p_1, p_{k+1}, \dots, p_n)$$

Calcoliamo la differenza nei costi:

Definiamo:

- $C(\sigma)$ = somma dei tempi di completamento in σ
- l_1 = lunghezza di p_1
- l_k = lunghezza di p_k = lunghezza di $l_{\min}$

*Tempi di completamento in σ^**

$$\begin{aligned} C(p_1) &= l_1 \\ C(p_2) &= l_1 + l_2 \\ &\dots \\ C(p_k) &= l_1 + l_2 + \dots + l_k \\ C(p_{k+1}) &= l_1 + l_2 + \dots + l_k + l_{k+1} \\ &\dots \end{aligned}$$

Tempi di completamento in σ' :

$$\begin{aligned} C'(l_{\min}) &= l_k \\ C'(p_2) &= l_k + l_2 \\ &\dots \\ C'(p_1) &= l_k + l_2 + \dots + l_{k-1} + l_1 \\ C'(p_{k+1}) &= l_k + l_2 + \dots + l_{k-1} + l_1 + l_{k+1} \\ &\dots \end{aligned}$$

Differenza nei costi:

I programmi $p_{k+1}, \dots, p_n$ hanno lo stesso tempo di completamento in entrambi gli ordini.

I programmi $p_2, \dots, p_{k-1}$ hanno la stessa differenza $(l_k - l_1)$ in σ' rispetto a σ^* .

La differenza totale è:

$$C(\sigma') - C(\sigma^*) = (k-1)(l_k - l_1)$$

Poiché $l_k = l_{\min} \leq l_1$ ($l_{\min}$ è il minimo), abbiamo:

$$l_k - l_1 \leq 0$$

Quindi:

$$C(\sigma') - C(\sigma^*) = (k-1)(l_k - l_1) \leq 0$$

Ovvero $C(\sigma') \leq C(\sigma^*)$.

Poiché σ era ottimo, deve valere $C(\sigma') = C(\sigma)$, quindi anche σ' è ottimo.

Conclusione: Esiste sempre una soluzione ottima che inizia con il programma di lunghezza minima. ✓

Correttezza dell'algoritmo greedy:

Per induzione:

1. La scelta greedy per la prima posizione è corretta (appena dimostrato)
2. Dopo aver rimosso il primo programma, il sottoproblema rimanente ha la stessa struttura
3. Per ipotesi induttiva, ordinare i rimanenti programmi per lunghezza crescente è ottimo

Quindi l'algoritmo greedy è corretto. ✓

Esercizio 6.2 - Selezione Attività (9 punti)

Consegna: Si considerino n attività $\{a_1, a_2, \dots, a_n\}$, ciascuna caratterizzata da tempo di inizio s_i e tempo di fine f_i (con $s_i < f_i$). Due attività a_i e a_j sono compatibili se i loro intervalli temporali non si sovrappongono ($f_i \leq s_j$ oppure $f_j \leq s_i$).

Vogliamo selezionare un insieme massimale di attività mutualmente compatibili.

(a) Scrivere il codice di un algoritmo greedy che risolve il problema in tempo lineare, assumendo che le attività siano già ordinate per tempo di fine crescente ($f_1 \leq f_2 \leq \dots \leq f_n$).

(b) Dimostrare la proprietà di scelta greedy per l'algoritmo del punto (a).

Soluzione:

Parte (a): Algoritmo Greedy

Scelta greedy: Seleziona sempre l'attività compatibile che termina prima.

Pseudocodice:

```

1  ActivitySelection(s[1..n], f[1..n])
2    // Input: s[i] = tempo inizio, f[i] = tempo fine
3    // Precondizione: f[1] ≤ f[2] ≤ ... ≤ f[n]
4
5    A = {a1}                // Inizia con la prima attività
6    lastFinish = f[1]
7
8    for i = 2 to n do
9        if s[i] ≥ lastFinish    // Compatibile con l'ultima selezionata
10            A = A ∪ {ai}
11            lastFinish = f[i]
12
13    return A

```

Analisi di Complessità:

- Inizializzazione: $\Theta(1)$ (linee 5-6)
- Loop principale: $\Theta(n)$ (linee 8-11)
 - Ogni iterazione: $\Theta(1)$

Complessità totale: $\Theta(n)$

Nota: Se le attività non fossero ordinate, dovremmo prima ordinarle in $\Theta(n \log n)$.

Parte (b): Dimostrazione Proprietà Scelta Greedy

Teorema (Scelta Greedy): Esiste una soluzione ottima che contiene l'attività a_1 (quella che termina per prima).

Dimostrazione (Exchange Argument):

Sia $A^* = \{a_{i_1}, a_{i_2}, \dots, a_{i_k}\}$ una soluzione ottima qualunque, con attività ordinate per tempo di fine ($f_{i_1} \leq f_{i_2} \leq \dots \leq f_{i_k}$).

Caso 1: Se $i_1 = 1$, allora $a_1 \in A^*$, e abbiamo finito. ✓

Caso 2: Se $i_1 \neq 1$, ovvero A^* non contiene a_1 .

Costruiamo $A' = \{a_1, a_{i_2}, \dots, a_{i_k}\}$ sostituendo a_{i_1} con a_1 .

Vogliamo dimostrare che A' è una soluzione valida e ottima.

A' è una soluzione valida?

Dobbiamo verificare che a_1 è compatibile con a_{i_2} :

- Per ipotesi, a_{i_1} e a_{i_2} sono compatibili, quindi $f_{i_1} \leq s_{i_2}$
- Poiché a_1 termina prima di tutte le altre attività: $f_1 \leq f_{i_1}$
- Quindi: $f_1 \leq f_{i_1} \leq s_{i_2}$

Dunque a_1 è compatibile con a_{i2} . ✓

Per transitività, a_1 è compatibile con tutte le attività $a_{i2}, \dots, a_{ik}$.

A' è ottima?

$|A'| = |A^*|$ (sostituiamo un'attività con un'altra), quindi A' contiene lo stesso numero di attività.

Poiché A^* era massimale, anche A' è massimale. ✓

Conclusione: Esiste sempre una soluzione ottima che contiene l'attività che termina per prima.

Correttezza dell'algoritmo:

Per induzione:

1. La scelta greedy del primo elemento è corretta (appena dimostrato)
2. Dopo aver scelto a_1 , il sottoproblema consiste nel selezionare attività che iniziano dopo f_1
3. Per ipotesi induttiva, la scelta greedy (prossima attività compatibile che termina prima) è ottima per il sottoproblema
4. Quindi l'algoritmo greedy produce una soluzione ottima. ✓

APPENDICE: TABELLA RIEPILOGATIVA COMPLESSITÀ

Algoritmi di Ordinamento

Algoritmo	Caso Migliore	Caso Medio	Caso Peggior	Spazio
InsertionSort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$
MergeSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
QuickSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(\log n)$
HeapSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(1)$

Strutture Dati

Operazione	BST (medio)	BST (peggiore)	Heap	Hash Table
Search	$\Theta(\log n)$	$\Theta(n)$	-	$\Theta(1)^*$
Insert	$\Theta(\log n)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)^*$
Delete	$\Theta(\log n)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)^*$
Find-Min	$\Theta(\log n)$	$\Theta(n)$	$\Theta(1)$	-

*caso medio con buona funzione hash

Master Theorem

Per $T(n) = aT(n/b) + f(n)$:

- **Caso 1:** $f(n) = O(n^{(\log_b a - \epsilon)}) \rightarrow T(n) = \Theta(n^{\log_b a})$
 - **Caso 2:** $f(n) = \Theta(n^{\log_b a}) \rightarrow T(n) = \Theta(n^{\log_b a} \log n)$
 - **Caso 3:** $f(n) = \Omega(n^{(\log_b a + \epsilon)})$ e regolarità $\rightarrow T(n) = \Theta(f(n))$
-

NOTE METODOLOGICHE

Struttura Standard di una Soluzione

Ogni esercizio richiede:

1. **Pseudocodice numerato con:**
 - Numerazione delle linee
 - Commenti chiari sulle invarianti
 - Gestione esplicita dei casi base
2. **Dimostrazione di Correttezza:**
 - Identificazione dell'invariante
 - Dimostrazione caso base
 - Dimostrazione passo induttivo
 - Conclusione esplicita
3. **Analisi di Complessità:**
 - Ricorrenza (se applicabile)
 - Applicazione Master Theorem o metodo di sostituzione
 - Distinzione tra caso migliore/medio/peggiore
 - Complessità finale in notazione Θ

Errori Comuni da Evitare

1. **Pseudocodice:**
 - Non mischiare sintassi di linguaggi diversi
 - Evitare dettagli implementativi irrilevanti
 - Usare sempre la numerazione delle linee
2. **Dimostrazioni:**
 - Non assumere ciò che si deve dimostrare
 - Esplicitare sempre i casi base
 - Non omettere passaggi "ovvi"
3. **Complessità:**

- Non confondere O , Ω e Θ
 - Ricordare di considerare spazio oltre al tempo
 - Verificare le precondizioni del Master Theorem
-

Fine Homework

Preparato seguendo la metodologia del corso di Algoritmi e Strutture Dati Università di Padova - Anno Accademico 2024/2025